

DA6401W - Assignment 2

Department of Data Science and AI, IIT Madras

Feb 2026

Topics covered: Multilayer perceptron, backpropagation, activation functions, gradient descent

Submit your solutions in one PDF file with all answers, codes, plots and outputs written/printed in that file.

1. Effect of Network Depth on Learning Performance

In this question, you will study how the depth of a neural network affects learning and generalization. You must implement all models **from scratch using NumPy only**. The use of sklearn, torch, or tensorflow for model implementation is strictly prohibited.

(a) Dataset Generation and Experimental Setup

Generate a synthetic binary classification dataset as follows:

- Input space: $x = (x_1, x_2) \in \mathbb{R}^2$
- Number of samples: $N = 1000$
- Class labels: $y \in \{0, 1\}$

Data generation rule:

$$y = \begin{cases} 1, & \text{if } (x_1^2 + x_2^2) > 0.5 \\ 0, & \text{otherwise} \end{cases}$$

where (x_1, x_2) are sampled uniformly from $[-1, 1] \times [-1, 1]$.

Split the dataset into:

- Training set: 80% of the data
- Test set: 20% of the data

Plot the dataset and explain why a linear classifier cannot solve this problem.

(b) Neural Network Architectures

Implement the following two neural networks:

Model 1 (Shallow Network):

- Input layer: 2 neurons
- One hidden layer: 20 neurons

- Output layer: 1 neuron

Model 2 (Deeper Network):

- Input layer: 2 neurons
- Hidden layer 1: 10 neurons
- Hidden layer 2: 10 neurons
- Output layer: 1 neuron

Both models must use:

- ReLU activation in hidden layers
- Sigmoid activation in the output layer

(c) Training Procedure

Train both models using the following fixed hyperparameters:

- Loss function: Binary Cross-Entropy
- Optimization method: Batch Gradient Descent
- Learning rate: $\eta = 0.01$
- Number of epochs: 100
- Weight initialization: $\mathcal{N}(0, 0.01)$
- Bias initialization: zeros

Plot the training loss versus epochs for both models on the same graph.

(d) Evaluation Metrics

Evaluate both trained models on the test set using:

- Classification accuracy
- Confusion matrix

Use a decision threshold of 0.5 on predicted probabilities.

(e) Analysis

Compare the two models and discuss the effect of depth in terms of:

- Representational capacity
- Optimization difficulty
- Generalization performance

2. Backpropagation in a Feedforward Neural Network

Consider a fully connected feedforward neural network with one hidden layer used for a regression task. The network structure is as follows:

- **Input layer:** $x \in \mathbb{R}^d$
- **Hidden layer:**

$$h = \sigma(W^{(1)}x + b^{(1)})$$

- **Output layer:**

$$\hat{y} = W^{(2)}h + b^{(2)}$$

where $\sigma(\cdot)$ is the **ReLU activation function**. The loss function used is the **mean squared error (MSE)** defined as:

$$L = \frac{1}{2} (y - \hat{y})^2$$

(a) **Forward Pass**

Derive the expression for the network output $\hat{y}$ in terms of the input x , weights, and biases.

(b) **Loss Gradient at Output Layer**

Compute the gradient of the loss L with respect to the output layer parameters $W^{(2)}$ and $b^{(2)}$.

(c) **Backpropagation Through Hidden Layer**

Using the chain rule, derive the expression for the gradient of the loss with respect to the hidden layer weights $W^{(1)}$. Clearly indicate how the derivative of the ReLU activation affects the gradient flow.

(d) **Role of the Chain Rule**

Explain briefly how the chain rule enables efficient gradient computation in deep networks and why backpropagation is computationally efficient compared to naïve differentiation.

(e) **Vanishing Gradient Discussion**

State whether the use of ReLU activation helps mitigate the vanishing gradient problem. Justify your answer.

3. Simple Gradient Descent

You are optimizing a loss function $\mathcal{L}(w) = w^4 - 10w^2 + 5w + 40$.

1. Start with an initial parameter $w_0 = 1$. Using a learning rate (step size) of $\eta = 0.01$, perform one manual iteration of gradient descent to find w_1 . Calculate the squared error for w_0 and w_1 . Did the loss decrease?
2. Will this setup of initial parameter $w_0 = 1$ reach global minima of the given error surface? Give the range of the initial parameter that will definitely converge to the global minima in the standard gradient descent approach.
3. If you chose a learning rate of $\eta = 2.0$ for this specific problem, what would happen to w_1 and the loss at w_1 ? Show the calculation.
4. Plot the loss function for the range $w \in [-3, 3]$

4. Code the Activation Functions

1. Create a class or set of functions in Python (using numpy) for the following activation functions: Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$ Tanh: $\tanh(x) = \frac{1-e^{-2x}}{1+e^{-2x}}$ ReLU: $\max(0, x)$ Leaky ReLU: x if $x > 0$, else αx (use $\alpha = 0.01$).
2. Implement the derivative for each function.

3. Visualization: Generate a range of inputs from -5 to 5. Plot each activation function and its corresponding derivative on the same graph.

5. Numerical: Forward Pass in a Multilayer Perceptron

Consider a multilayer perceptron with:

- Input: $x = [1, -2]^T$
- One hidden layer with 2 neurons and ReLU activation
- Output layer with 1 neuron (linear activation)

The parameters are given as:

$$W^{(1)} = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$W^{(2)} = [2 \quad -1], \quad b^{(2)} = 0$$

- (a) Compute the hidden layer pre-activation values.
- (b) Apply the ReLU activation.
- (c) Compute the final network output.

6. Numerical Verification of Universal Approximation Theorem

A single hidden layer multilayer perceptron (MLP) with a non-linear activation function can approximate any continuous function on a compact domain.

Consider the target function:

$$f(x) = \sin(2\pi x) + 0.5x^2, \quad x \in [0, 1]$$

A single hidden layer neural network is defined as:

$$\hat{f}(x) = \sum_{i=1}^3 \alpha_i \sigma(w_i x + b_i)$$

where the activation function is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The network parameters are:

$$w = [2, -3, 4], \quad b = [-1, 2, -2], \quad \alpha = [1.5, -1, 0.5]$$

- (a) Compute the network output $\hat{f}(x)$ for the inputs: $x = \{0, 0.5, 1\}$

(b) Compute the Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{3} \sum_{i=1}^3 \left(f(x_i) - \hat{f}(x_i) \right)^2$$

7. Programming: Gradient Checking for Neural Network Training

you need to implement a gradient checking procedure to verify the correctness of backpropagation in a multilayer perceptron (MLP).

Consider the following neural network architecture for regression:

- Input layer: $x \in \mathbb{R}^2$
- Hidden layer: 5 neurons with sigmoid activation
- Output layer: 1 neuron with linear activation

The network equations are:

$$h = \sigma(W^{(1)}x + b^{(1)})$$

$$\hat{y} = W^{(2)}h + b^{(2)}$$

The loss function is:

$$L = \frac{1}{2}(y - \hat{y})^2$$

You must generate a small synthetic dataset consisting of 20 samples where:

$$x_1, x_2 \sim \text{Uniform}(-1, 1)$$

and the target output is:

$$y = x_1^2 + x_2^2$$

- Implement forward propagation and manual backpropagation using NumPy.
- Implement gradient checking using finite difference approximation:

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta + \epsilon) - L(\theta - \epsilon)}{2\epsilon}$$

where $\epsilon = 10^{-5}$ and θ represents any network parameter.

- Compare analytical gradients obtained using backpropagation with numerical gradients for at least three randomly chosen parameters.

- (d) Report the relative error between analytical and numerical gradients and comment on the correctness of your implementation.

Implementation Requirements:

- Use NumPy only.
- Initialize weights randomly.
- Perform gradient checking before training the network.

8. **Programming:** Consider the problem of function approximation under different hypothesis classes.

Let the true data-generating function be

$$y = \sin(x),$$

where $x \in [0, 2\pi]$. A dataset is created by sampling n points uniformly from this interval and adding i.i.d. Gaussian noise:

$$y_i = \sin(x_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2).$$

You are asked to study the **bias–variance tradeoff** using different hypothesis classes.

Let $\hat{f}_{\mathcal{D}}(x)$ denote the predictor learned from a training dataset $\mathcal{D}$.

The expected prediction at a point x , over all possible training datasets of size n , is given by

$$\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)].$$

The **bias** of the estimator at x is defined as

$$\text{Bias}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)] - \sin(x).$$

The **variance** of the estimator at x is defined as

$$\text{Var}(x) = \mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}_{\mathcal{D}}(x) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)] \right)^2 \right].$$

1. **Linear Hypothesis**

Assume the hypothesis class is linear:

$$\hat{y} = Wx + c.$$

- (a) Fit the model to the sampled data.
- (b) Report the training & testing error. Visualize the learned hypothesis along with the true function and sampled points.
- (c) Experiment with different values of W and c (either manually or via least-squares fitting).

(d) Comment on the bias and variance of this model with respect to the true function.

2. Polynomial Hypothesis

Now consider a polynomial hypothesis of degree d :

$$\hat{y} = \sum_{k=0}^d w_k x^k.$$

- (a) Fit polynomial models for increasing values of d (e.g., $d = 3, 5, 10$).
- (b) Report the training & testing error. Visualize the fitted polynomials along with the true function and sampled data.
- (c) Observe the behavior of the learned function as the degree increases.
- (d) Comment on how bias and variance change as model complexity increases.

3. Generalization Behavior (Optional)

Repeat the above experiments using different random samples from the same data-generating process. Compare training and test performance across models and comment on generalization.

9. Consider a Multilayer Perceptron (MLP) consisting of fully connected layers with an activation function applied after each linear transformation.

- 1. Explain why activation functions are essential in an MLP. What would be the representational power of an MLP if no activation functions were used?
- 2. Let an MLP consist of multiple layers, each of the form

$$\mathbf{h}^{(l)} = \phi(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}),$$

where $\phi(\cdot)$ is an activation function. Show that if $\phi(\cdot)$ is the identity function, the entire network reduces to a single linear transformation.

- 3. Discuss the role of nonlinearity in enabling MLPs to approximate complex functions. Briefly relate your answer to the Universal Approximation Theorem.
- 4. Compare sigmoid, tanh, and ReLU activation functions in terms of:
 - (a) output range,
 - (b) gradient behavior,
 - (c) suitability for deep networks.

10. Consider a binary classification problem with input-label pairs

$$x \in \mathbb{R}^d, \quad y \in \{0, 1\}.$$

A fully connected neural network with an input dimension of d is trained using two hidden layers of sizes $\frac{d}{2}$ and $\frac{d}{4}$, followed by a single output neuron where all hidden layers use sigmoid activation functions. The output layer uses a sigmoid activation, and the loss function is binary cross-entropy.

The network is trained using gradient-based optimization under the following initialization and training settings:

1. All weights and biases are initialized to the same constant value.
 2. Xavier initialization is used for all layers.
 3. Weights are initialized randomly with small independent noise.
 - (a) For each initialization setting, determine whether the network is able to learn meaningful representations.
 - (b) For each case, explain the effect of the initialization on symmetry breaking and gradient flow during training.
11. Implement a two-layer neural network with 2 input neurons, 10 hidden neurons, and 1 output neuron using ReLU activation in the hidden layer to solve the XOR classification problem. Investigate how the learning rate and bias initialization affect the dying ReLU phenomenon. Weights are initialized as

$$W_{ij} \sim \mathcal{N}(0, 0.01)$$

unless stated otherwise.

Tasks

1. Train the network under the following five configurations:
 - Case 1: Learning rate = 0.1, Bias initialization = -5.0, Epochs = 500
 - Case 2: Learning rate = 0.1, Bias initialization = 0.0, Epochs = 500
 - Case 3: Learning rate = 0.01, Bias initialization = -5.0, Epochs = 500
 - Case 4: Learning rate = 0.01, Bias initialization = 0.0, Epochs = 500
 - Case 5: Learning rate = 0.01, Bias initialization = random, Epochs = 10000
2. For each configuration, plot a grid of figures showing:
 - Row 1: Training loss versus epochs
 - Row 2: Fraction of dead hidden neurons versus epochs
 - Row 3: Classification accuracy versus epochs
3. Answer the following questions:
 - (a) Which configurations exhibit dying ReLU behavior? Explain why.
 - (b) Why are dead ReLU neurons unable to recover through gradient descent?

12. Basics of Computational Graphs

Consider the function

$$z = (x + y) y$$

where $x, y \in \mathbb{R}$.

(a) **Graph Representation**

Write the sequence of intermediate computations needed to evaluate z . (These define the computational graph nodes.)

(b) **Forward Evaluation**

Compute z for:

$$x = 2, \quad y = 3$$

(c) **Gradient Computation**

Using the computational graph, compute:

$$\frac{\partial z}{\partial x}, \quad \frac{\partial z}{\partial y}$$

(d) **Conceptual**

In one or two sentences, explain why computational graphs are useful for training neural networks.